Increasing Test Coverage by Automating BDD Tests in Proofs of Concepts (POCs) using LLM

Shexmo Richarlison Ribeiro dos Santos
Federal University of Sergipe
São Cristóvão, Brazil
shexmor@gmail.com

Raiane Eunice S. Fernandes
Federal University of Sergipe
São Cristóvão, Brazil
raiane.fernandes2199@gmail.com

Marcos Cesar Barbosa dos Santos
Federal University of Sergipe
São Cristóvão, Brazil
marcos.cesar.se@gmail.com

Michel S. Soares
Federal University of Sergipe
São Cristóvão, Brazil
michel@dcomp.ufs.br

Fabio Gomes Rocha
ISH (SafeLabs)
Vitória, Espírito Santo, Brazil
Federal University of Sergipe
São Cristóvão, Brazil
gomesrocha@gmail.com

Sabrina Marczak
School of Technology PUCRS
Porto Alegre, Rio Grande do Sul
Brazil
sabrina.marczak@pucrs.br

Abstract

In today's landscape, software manufacturers must deliver quickly and with high quality to remain competitive, especially in the cybersecurity sector. Recognizing this need, we have recently implemented several strategies to accelerate our time to market without compromising quality. We introduced the Proof of Concept (POC) and Proof of Value (POV) stages in the Enterprise Architecture team before initiating inception processes for Minimum Viable Products (MVP) and new features. Initially, these proofs focused only on concept validation. However, since 2023, due to the growing need for rapid and high-quality innovation, POCs/POVs have begun to include robust implementations. We adopted the Behaviour-Driven Development (BDD) approach to define user stories and acceptance criteria, which provided a solid evaluation of POC/POV quality and involved the implementation team from the outset. To prevent products from incurring technical debt, we implemented AutoDevSuite, which uses LLM to generate tests based on user stories and acceptance criteria automatically. We used AutoDevSuite in a POC/POV of a cybersecurity product, and the results showed a significant expansion in test coverage, aligned with the acceptance criteria, demonstrating the tool's effectiveness in automating and improving test quality.

CCS Concepts

• **Software and its engineering → Software design techniques**.

SBQS 2024, November 05–08, 2024, Salvador, Brazil

ACM ISBN 979-8-4007-1777-2/24/11
https://doi.org/10.1145/3701625.3701637

Keywords

Artificial Intelligence (AI), Automatic test code generator, Behavior-Driven Development (BDD), Large Language Model (LLM), Software quality

ACM Reference Format:
Shexmo Richarlison Ribeiro dos Santos, Raiane Eunice S. Fernandes, Marcos Cesar Barbosa dos Santos, Michel S. Soares, Fabio Gomes Rocha, and Sabrina Marczak. 2024. Increasing Test Coverage by Automating BDD Tests in Proofs of Concepts (POCs) using LLM. In *XXIII Brazilian Symposium on Software Quality (SBQS 2024), November 05–08, 2024, Salvador, Brazil.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3701625.3701637

1 Introduction

In the software industry, especially in critical areas such as cybersecurity, constant innovation is a must. Gartner [13] reports that strategic trends in cybersecurity are continually adjusted and expanded to meet new protection needs. This requires agile deliveries without compromising quality, as evidenced by the CrowdStrike incident in July 2024, which resulted in a cyber blackout.

Recognizing the need to balance innovation and quality, we have recently implemented several strategies to speed up time to market without compromising excellence. We introduced the Proof of Concept (POC) and Proof of Value (POV) stages in the Enterprise Architecture team, which were carried out before the inception phase to create Minimum Viable Products (MVP) and innovative functionalities. This model is similar to the one proposed by Berenguer [3], which combines the lean startup approach with research methodologies, where a POV follows a POC and, if approved, evolves into an existing product functionality or an MVP.

Initially, POCs and POVs focussed solely on validating concepts and determining their value to the business. However, due to the need for agile deliveries, since 2023, POCs/POVs have included more robust implementations to facilitate the development of the MVP, should the proof be approved. The architecture team has adopted an approach that begins with Behaviour-Driven Development (BDD) to define requirements through user stories and their acceptance scenarios. These requirements consider a POC/POV completed, providing a more solid and standardised assessment.

However, the tests were not automated, and the responsibility for the final evaluation of the products rests with the quality team.

While the absence of testing may be tolerable in the concept validation phase, it becomes critical when these tests progress to functional components. In such cases, development teams often need help implementing automated tests for BDD acceptance scenarios while integrating new components, resulting in technical debts that are difficult to address in the future. To avoid creating new products with technical debts and guarantee the quality of POCs/POVs, we implemented AutoDevSuite. This tool uses Large Language Models (LLMs) to generate automated tests based on user stories and their acceptance criteria.

AutoDevSuite was applied in a proof of concept at a cybersecurity software company during the early stages of development, a context that has yet to be explored [19]. The team responsible for creating new POCs/POVs identified that the previous POCs/POVs had less than 30 per cent test coverage. The aim was to achieve over 60 per cent coverage by adopting the tool.

This paper reports on the experience of adopting AutoDevSuite in a Proof of Concept/Proof of Value (POC/POV) for a Web Data Collection System. We present the current POC/POV development state and the results obtained with AutoDevSuite. The main lessons learned from this process are the agility in test generation and the importance of having experienced personnel to avoid introducing faulty components into the system.

This paper is structured as follows: Section 2 presents the concepts related to proof of concept and the Behaviour-Driven Development (BDD) framework. Section 3 discusses studies related to our research. Section 4 contextualises this study in the context of software development. Section 5 describes the methodological steps adopted to achieve the results, which are discussed in Section 6; Finally, Section 7 presents the main contributions of this study and suggests directions for future work.

2 Background

The following discusses concepts inherent to proof of concept, Behaviour-Driven Development (BDD), Test Coverage, and the large language model (LLM), the framework used to carry out this research, and the artificial intelligence used to automatically generate test codes, respectively.

2.1 Proof of concept

Proof of concept (POC) demonstrates a product, service or solution in a given context to verify whether it meets the requirements and delivers value [1]. POC is a strategic tool used to reduce costs associated with investments in solutions or developments that may not meet requirements or that do not generate expected value. They offer several benefits, including evaluating compliance with requirements, presenting results to interested parties, and identifying problems and possible barriers in solution development [5].

Berenguer [3] presents a model in which proof of concept is used to align potential demand, testing hypotheses related to the value proposition. In this context, the Minimum Viable Product (MVP) appears as an evolution of a successful POC that presents a solid value proposition. Thus, POC is a crucial initial phase for product development, allowing for preliminary validation before moving on to more intensive and expensive development phases.

For a proof of concept to be effective, it must have well-defined requirements. Clarity in requirements facilitates technical validation and ensures that all stakeholders are aligned on project objectives and expectations. In this sense, Behavior-Driven Development (BDD) can be highly beneficial. By promoting ubiquitous language and clear, behaviour-based specifications, BDD ensures that POC requirements are understandable and verifiable, providing a solid foundation for evolution into an MVP.

2.2 Behavior-Driven Development

Behavior-Driven Development (BDD) evolved from Test-Driven Development (TDD) to answer the fundamental questions of where to start, what to test, what not to test, when to test, how to name tests, and why a test fails. BDD describes system behaviour from the perspective of stakeholders at all levels of granularity, which improves communication and understanding of the system [12]. In this way, BDD offers a comprehensive view of agile analysis and automated testing, helping teams build, deliver value, and ensure software quality [15].

BDD uses a ubiquitous language accessible to all team members, automates acceptance tests based on the criteria provided, and promotes readable code writing that reflects the objectives of the BDD scenarios [17]. This approach not only facilitates understanding of how the software works but also formalizes understanding and guides development based on the automation of acceptance tests, ensuring the validity and consistency of the developed functionalities.

The BDD cycle begins with meetings between stakeholders and customers to discuss the business vision and define usage scenarios in the format of user stories [15]. These scenarios, structured in the *Gherkin* language, guide the development and automation of acceptance tests. Scenario documentation formalizes understanding and guides development, ensuring consistent and validated releases.

2.3 Test coverage

Test coverage is an essential metric in software development that indicates how much a program's source code is exercised through tests [16]. This metric covers several aspects, such as the percentage of lines of code, branches and paths executed during the execution of tests [4]. Test coverage does not replace debugging; it is a separate process focused on correcting errors.

High test coverage ensures that most of the software's functionalities has been verified, helping to identify and fix errors before the product is released into production. This practice not only improves the quality of the software but also increases confidence in its use [16]. In the context of BDD, high test coverage aims to ensure that the functionalities described in the user stories and acceptance criteria are validated, which helps to guarantee that the developers understand and implement the requirements correctly. On one hand, high test coverage contributes to a better understanding and validation of functionality; on the other hand, debugging is a separate process for resolving problems encountered.

To make this assessment possible, there are several plugins which automate and generate coverage reports. This report used *pytest-cov*,

which uses the coverage tool to record and produce coverage reports and can even show which lines of a given file were not covered [6]. In this way, this tool contributes to creating test routines that cover all parts of the program, thus guaranteeing quality of the final product.

2.4 Large Language Models

Over the past two years, Large Language Models (LLMs) have experienced significant growth in usage due to their ability to encode vast amounts of knowledge from textual data while understanding the context of the processed information [19]. This characteristic makes them particularly suitable for applications in test automation and improving test coverage.

LLMs differ from other generative Artificial Intelligence (AI) due to their extensive base of parameters that are appropriately structured and updated with user feedback. This aspect allows good results in responding to prompts provided in natural language.

Regarding their architecture, the most popular LLMs have their model based on transformers [19], which analyze grammatical structures by examining the relationship of words in a given context.

LLMs can generate queries, codes, and other relevant information from structured prompts, demonstrating similarities to human-generated queries [2]. In the context of test automation, LLMs can create test cases based on bug reports, which allows developers to increase efficiency by automating the reproduction of failures and bugs [7, 19].

Furthermore, LLMs can enhance test case preparation and other tasks related to test automation, thus expanding the coverage of test scenarios [19]. Therefore, using LLMs facilitates the identification and correction of problems and contributes to greater efficiency in the software development process.

3 Related Work

Some work that relate to what will be covered in this study are briefly discussed as follows.

Lee, Gong, and Cao [9] demonstrate a generative AI approach to translating human language into high-level programming language.

Mock, Melegati, and Russo [11] introduce the approach that proposes TDD automation through Generative AI.

Karpurapu et al. [8] evaluate a detailed approach focused on improving BDD practices through LLMs for the automatic generation of acceptance tests.

Takerngsaksiri et al. [18] present and evaluate *PyTester* as a tool for generating formal test cases from natural language.

Schafer et al. [14] exploit Large Language Models (LLMs) for the automatic generation of unit tests without the need for additional training. Using OpenAI's gpt3.5-turbo LLM, TESTPILOT was developed for JavaScript, generating unit tests that achieved 70.2% line coverage and 52.8% branch coverage. These results surpass conventional techniques such as the feedback-based approach, Nessie, showing that the quality of tests also depends on the integrity of the prompts provided to LLM.

Lu et al. [10] highlight LLaMA-Reviewer, an innovative framework that automates code review using LLaMA, a popular LLM. With efficient parameter fine-tuning (PEFT), the system achieves performance comparable to existing models, using less than 1% of the trainable parameters. Experiments reveal the importance of input representation and PEFT methods, with all components available as open-source code to drive advances in the area.

When using BDD scenarios to generate test codes automated by LLMs, it will be possible to present how the codes performed according to the analyses carried out to contribute to the related work presented. We will be able to advance studies regarding using LLMs and how they can contribute to software development.

4 Contextualization

In 2024 alone, the team demonstrated its adaptability by studying and implementing four POCs/POVs, three of which became functionalities for existing products and one of which, once approved, progressed to the inception stage to become an MVP. These initiatives significantly influenced product development. The products are developed using various technology stacks. Generally, the development of POCs is started in Python, but the most suitable stack for the product is also studied during the value analysis. As explained in the introduction, before the start of each POC, detailed documentation was carried out that generates user stories and their acceptance scenarios, allowing an assessment to be made of whether the POC/POV has been successfully completed and whether it meets the basic requirements, providing a path for the team to finalise the study.

The company's model seeks agility due to the required time to market without compromising product quality. The typical process sequence is as follows, as illustrated in Figure 1.

- Demand: Receiving demand from teams and managers.
- Base Documentation: Generation of user stories and acceptance criteria.
- Research: Initial investigation and definition of technical requirements.
- Preparation of the POC: Development of the proof of concept.
- Validation of Resources for POV: Assessment of POC in environments similar to the real one.
- Presentation: Demonstration of results to stakeholders.
- Decision: If approved, delivery to the existing team or creation of a new team.

Initially, the concern when creating POCs/POVs was to verify technical feasibility, focusing mainly on documentation rather than implementation. However, from 2023 onwards, due to the increased complexity of POCs/POVs, the constant need for innovation and time to market, it has become necessary to validate them in environments that simulate actual conditions of use. This has resulted in a greater emphasis on the functional delivery of the POC/POV. Thus, faced with this scenario, the need arose to develop a strategy allowing the delivery of POCs/POVs that could become high-quality minimum viable products (MVPs) without burdening the team with technical debts related to testing. The central question was how to maintain agility in the preparation of POCs/POVs while at the same time guaranteeing more excellent test coverage in the codes generated.

The solution proposed to increase test automation and test coverage without significantly impacting the current process was to draw up standard prompts for test generation based on user stories and acceptance scenarios. Thus, a system was developed that can

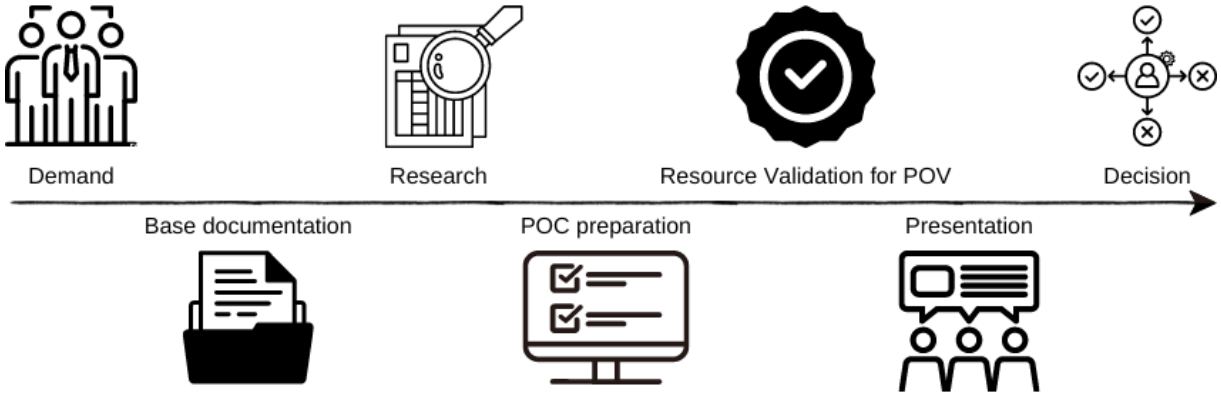

Figure 1: Process adopted by the company

be used by programmers, testers, or requirements analysts. This system takes user stories and acceptance scenarios as input and, based on standard prompts, automatically generates the necessary tests.

The system developed is available as open-source software under the name AutoDevSuite and can be accessed via the link[1]. This solution is a contribution from the project and the company to the community, aimed at improving quality and efficiency in software development.

AutoDevSuite was used in a POC/POV to analyse the ability to increase the amount of test automation implemented based on acceptance scenarios, focusing on test coverage.

5 Methodology

POCs transformed into features often become untested, generating technical debt that is frequently unresolved since new functionalities are constantly prioritized. In this way, adopting LLMs can assist in the POC preparation phase in creating tests, with the test-first principle, improving coverage and ensuring more excellent quality of the future feature.

This study was conducted in collaboration with the Enterprise Architecture team, responsible for carrying out Proofs of Concepts (POCs). POCs carried out in 2024 were analyzed using the pytest-cov tool to evaluate the tests' coverage. A specific POC, starting in February 2024, was selected for testing. This POC involves the development of a web-based data collector intending to achieve a minimum coverage of 60%. POC does not include a front-end, focusing only on the data collection bot. The implementation used were FastAPI, httpx, Beautiful Soup, Pytest, pytest-cov and pytest-bdd. For data management, we used sqlmodel with PostgreSQL as the database. The application was containerized using Docker and made available in a Kubernetes environment.

The Product Owner (PO) initially described the desired features of the product through user stories. Acceptance scenarios were developed with the team following the Gherkin pattern and registered on the Azure DevOps board. The POC development team, consisting of an Expert Data Engineer, two Full Developers, and an Expert Software Architect, refined the acceptance stories and scenarios in collaboration with the PO, adding test data to the scenarios.

The team generated automated tests in a one-shot model without additional interaction with LLM using user stories and acceptance scenarios. The team integrated the generated tests into the BDD automation cycle, ensuring each test covered the defined acceptance scenarios. During development, the team requested new inputs from LLM if any refactoring was necessary. At the end of the POC, a coverage test was carried out to identify the percentage of coverage achieved.

These steps were strictly followed to guarantee the quality and scope of automated tests, contributing to the development process' efficiency and the final product's robustness.

5.1 Implemented solution

The developed solution consists of a system that automates the generation of software tests based on user stories and their acceptance criteria. The system architecture is based on integrating several technologies, aiming to optimise the software development process, making it more efficient and accurate.

5.1.1 User interface (Streamlit): The point of interaction between the user and the system is an intuitive interface developed using the Streamlit library. Through this interface, the user enters relevant information for generating tests and documentation, such as:

- User story: Concise description of the functionality to be implemented from the end user's point of view.
- Acceptance criteria: Conditions must be met for the user story to be considered complete.
- Programming language: Language used in software development.
- Framework: Software structure used in the project.

5.1.2 Natural Language Processing (LangChain and LLMs): After collecting the information the user provides, the system uses the LangChain library to build a structured prompt, which will be sent

[1]https://autodevsuite-bdd.streamlit.app/

to an LLM, such as Gemini. The prompt includes the user story information, acceptance criteria, programming language and framework. The LLM processes the prompt and generates the following output:

- Tests: Automated test code specific to the programming language and framework provided.
- Explanation of how to use: Detailed instructions on how to use the implemented functionality.

5.1.3 Storage and Refinement (Memory): The results generated by the LLM are stored in memory, allowing the user to refine the information provided and redo the request if necessary. This functionality enables an iterative process of improving tests and documentation. Thus, the system produced has a cycle, shown in Figure 2.

Figure 2 illustrates the integration of the AutoDevSuite tool with the company's process. The integration involves synchronizing the automated tests generated by AutoDevSuite with the company's existing workflow, with continuous interaction with the LLM to ensure that acceptance criteria are met efficiently.

The process begins with the input data provided by the user, which includes the user story, acceptance criteria, programming language and framework. An LLM then processes this data. In the case of this study, Google's Gemini model was used, although alternatives such as Ollama, ChatGPT, and other similar models can also be used. After processing, the LLM generates the output delivered to the tool user.

6 Results and Discussion

The study is an experience report based on adopting the company's model, focusing mainly on expanding test coverage. At the time of the study, there was no specific focus on the quantitative comparison of effectiveness between LLMs and traditional testing methods. However, the expansion of test coverage was effectively observed, and it is recommended that a more in-depth study be carried out to obtain detailed comparative quantitative metrics.

Before implementing the automatic test generation tool using Large Language Models (LLMs), code coverage in our project was limited, reaching a maximum of 30%. This low coverage indicated that a significant portion of the code was not being tested properly, which could lead to software quality and maintainability issues.

Regarding the defects revealed, the focus of BDD and TDD is "test-first", i.e., tests guide development. We noticed that when dealing with large volumes of data, especially in cases of data ingestion, there was a need for more agile reconfiguration, allowing concepts to be validated in the initial phase before implementation. For example, in one of the user stories related to the volume of data in OpenSearch, we could anticipate infrastructure needs even before implementation began. We have not seen a drop in effectiveness; on the contrary, automation has contributed to greater efficiency on the part of the DevOps team when deploying resources. No significant impacts on defect detection were identified during the adoption phase.

With the introduction of the automatic test generation tool using LLMs, we observed a substantial increase in code coverage. Table 1 presents the detailed results by project package.

Package	Total Lines	Lines Covered	Line Rate (%)
app	144	142	98,61
app.data	150	150	100,00
app.infra	250	200	80,00
app.model	100	100	100,00
app.router	290	290	100,00
app.service	150	150	100,00
app.domain	622	622	100,00

Table 1: Code coverage results for new POC

The analysis of the results shows that the implementation of the LLM tool resulted in a substantial improvement in code coverage, going from a maximum of 30% to significantly higher values in all packages:

- **Packages with 100% Coverage:** The *app.data*, *app.model*, *app.router*, *app.service* and *app.domain* packages achieved 100% coverage, indicating the effectiveness of the LLM tool in generating complete tests for these areas.
- **Packages with Partial Coverage:** The *app* and *app.infra* packages presented coverage of 98.61% and 80%, respectively, representing a significant improvement, although there is still room for further optimization.
- **Need for technical knowledge or code review:** Although automated test generation has proven to be efficient, we have observed that the quality of the results can be compromised by poorly crafted acceptance scenarios or the inherent complexity of some cases. Additionally, the version of the LLM training model can influence the accuracy of the generated tests, resulting in outdated or faulty versions. This is particularly problematic for less experienced developers, who may introduce defective components into the system, negatively impacting the final delivery. Therefore, it is essential to implement a code review stage supervised by more experienced developers. This review acts as a quality control mechanism, ensuring the automated tests are accurate and comprehensive. Senior developers can identify and correct potential flaws, enhance the robustness of acceptance scenarios, and ensure that the system remains stable and functional. This process of mentoring and review not only improves the quality of the final product but also contributes to the professional growth of junior developers, fostering a culture of continuous learning and collaboration within the team.

In addition to improving code coverage, a standard prompt (see Listing 1) was developed for generating tests, which can be used by both the specific tool and other LLMs available on the market. This prompt serves as a guide for the automatic creation of tests, ensuring consistency and comprehensiveness.

```
You are a test analyst, responsible for creating
    unit tests, based on the user story {us} and
    the acceptance criteria {ac} based on gherkin.
     Create unit tests for the programming
    language {lp} based on the framework {fw} and
    explain how to use the code in {lang}.
```

Listing 1: Test Generation Prompt

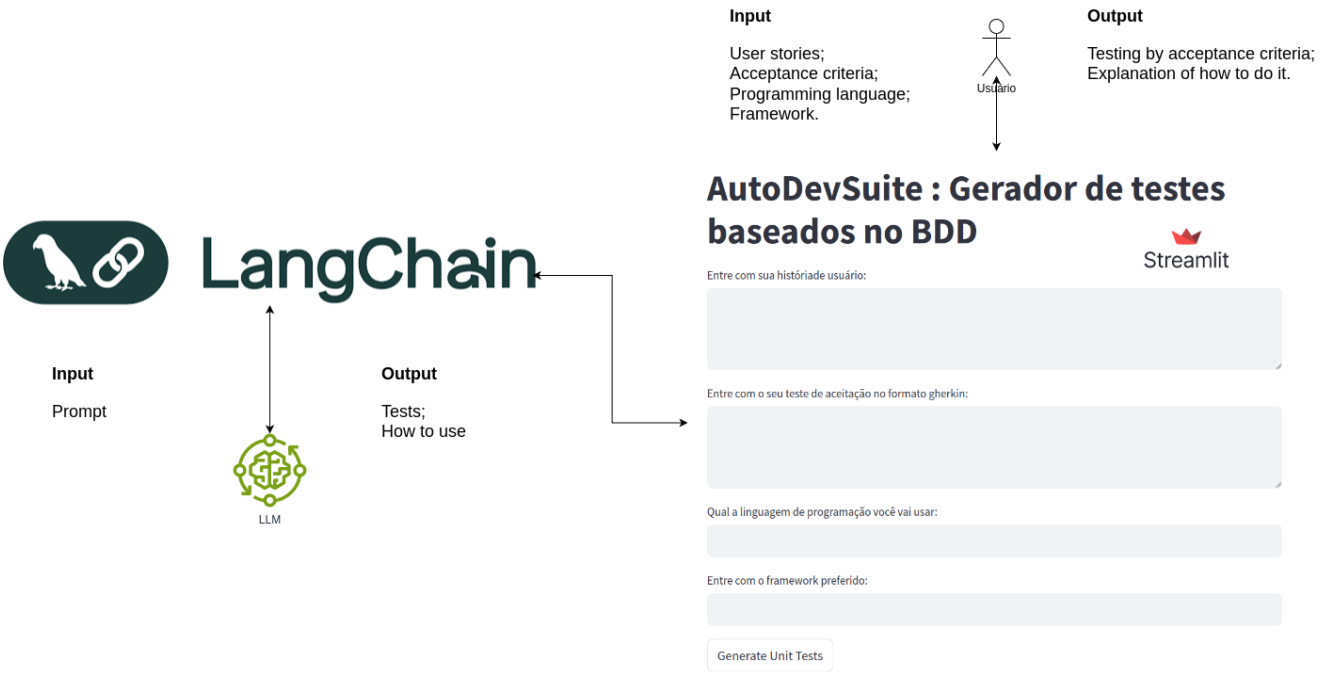

Figure 2: Automatic test generator based on user stories

We adopted the solution implemented in a Proof of Concept (POC) of a data collection system and obtained a significant increase in test coverage without compromising the POC delivery time. Unlike the work of Schafer et al. [14], validated only for JavaScript, and the work of Wannita et al. [18], restricted to use in Python, our approach is agnostic about the language and framework. Although we used Python with *pytest-bdd*, we validated our solution with *Behave*, demonstrating its versatility.

Our tool has the advantage of allowing simplified LLM switching and supporting both paid and free models. While Junyix et al.'s work [10] focuses on code review, our focus is exclusively on test automation, paving the way for future research that integrates both functionalities.

Therefore, the use of this solution provides several benefits to the software development process, including:

- Increased productivity: Automated test and documentation generation, freeing developers to focus on more complex tasks;
- Improved software quality: Automated tests ensure greater coverage and early error detection;
- Consistent and updated documentation: Automatically generated documentation accompanies software development, facilitating the understanding and use of the system;
- Flexibility: Possibility of using different LLMs and adapting the solution to the project's specific needs.

The implemented solution describes the potential of Artificial Intelligence in optimizing software development processes through the automated generation of tests and documentation. The combination of technologies such as Streamlit, LangChain and LLMs allows the creation of an efficient, flexible and easy-to-use system, which contributes to improving quality and productivity in software development, as shown in Figure 2.

The implementation of the automatic test generation tool brought to light several reflections and lessons learned:

- **Continuous Interaction with the Tool:** During the development cycle, we observed the need for continuous interactions with the LLM to generate tests. The TDD refactoring stage benefited significantly from the feedback from the LLM, allowing adjustments and improvements in the generated tests.
- **Support for the Development Team:** The tool proved to be an excellent support for the development team, automating repetitive tasks and allowing developers to focus on more critical aspects of the project. However, the team must have a good knowledge of the test implementation framework to maximize the benefits.
- **Quality of User Stories and Acceptance Scenarios:** Well-written user stories and acceptance scenarios are fundamental to the effectiveness of the LLM. Ambiguous or poorly written texts harm automatic code generation, highlighting the importance of clarity and precision in the requirements elicitation phase.

7 Conclusion

The proposed approach is based on a specific experience report, so we can only confirm its universal applicability in some domains. However, our practical experience demonstrates that the approach is adaptable to different technological stacks, as we have applied

the model in several languages, such as Python, C#, Java, Scala, and Rust. This adaptability suggests that the methodology can be adjusted to different software project contexts.

The main innovation of our approach lies in integrating Behavior-Driven Development (BDD) as a way of feeding LLM, which simplifies application in the context of the company. This simplification is designed to make the process more intuitive and user-friendly without negatively impacting agility or requiring extensive training. The distinctive advantage lies in compatibility with any framework or programming language, given that automation is centred on the context of the Gherkin, unlike previous methods that may have been limited to specific technologies or more rigid approaches.

Implementing the automatic test generation tool using LLMs significantly improved code coverage and development process efficiency. The reflections and lessons learned highlight the importance of continuous interaction with the tool and team knowledge of the test implementation framework.

We observed that well-written user stories and acceptance scenarios are crucial to maximise the effectiveness of LLM and that feedback during the refactoring stage of TDD can improve the generated tests, providing a more robust and efficient development cycle.

In addition, the tool will be made available as open source at the link[2], allowing other companies and researchers to adopt and improve the solution. Making the tool available as *open source* aims to foster collaboration and innovation in the software development community, facilitating the integration and use of LLMs in different contexts and frameworks.

Future research could explore the integration of code review with test automation, further expanding the benefits of these technologies in software development. This tool can be a valuable resource for development teams looking to automate test generation and improve code quality while saving time and resources. It could also include studies that explore different types of LLMs, scalability to larger projects, or integration with other software development tools.

Acknowledgments

Shexmo Santos would like to thank CAPES for the financial support (Bolsa de mestrado process no. 88887.888613/2023-00). Sabrina Marczak would like to thank CNPq for the financial support (Bolsa de Produtividade process no. 313181/2021-7).

References

[1] [n. d.]. Definition of Proof of Concept (POC) - Gartner Sales Glossary. https://www.gartner.com/en/sales/glossary/proof-of-concept-poc-

[2] Marwah Alaofi, Luke Gallagher, Mark Sanderson, Falk Scholer, and Paul Thomas. 2023. Can generative llms create query variants for test collections? an exploratory study. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1869–1873.

[3] Olimpia e Tomás David e Mazón Jose-Norberto Berenguer, Alberto e Alcaraz. [n. d.]. Da Pesquisa em Software Intensivo de Dados à Inovação em Espaços de Dados: Um Serviço de Pesquisa de Dados Tabulares. *Software IEEE* 41 ([n. d.]). https://doi.org/10.1109/MS.2024.3359333

[4] Armando Fox, David A Patterson, and Samuel Joseph. 2013. *Engineering software as a service: an agile approach using cloud computing*. Strawberry Canyon LLC.

[5] C. Gonzalez-Perez, B. Henderson-Sellers, T. McBride, G.C. Low, and X. Larrucea. 2016. An Ontology for ISO software engineering standards: 2) Proof of concept and application. *Computer Standards & Interfaces* 48 (2016), 112–123. https://doi.org/10.1016/j.csi.2016.04.007 Special Issue on Information System in Distributed Environment.

[6] Ionel. 2023. pytest-cov 5.0.0. Disponível em: https://pypi.org/project/pytest-cov/. Acesso em: 21 Junho 2024.

[7] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.

[8] Shanthi Karpurapu, Sravanthy Myneni, Unnati Nettur, Likhit Sagar Gajja, Dave Burke, Tom Stiehm, and Jeffery Payne. 2024. Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. *IEEE Access* (2024).

[9] Eric Lee, Jiayu Gong, and Qinghong Cao. 2023. Object Oriented BDD and Executable Human-Language Module Specification. In *2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD-Winter)*. IEEE, 127–133.

[10] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning. In *2023 IEEE 34TH INTERNATIONAL SYMPOSIUM ON SOFTWARE RELIABILITY ENGINEERING, ISSRE (Proceedings International Symposium on Software Reliability Engineering)*. IEEE; IEEE Comp Soc, Tech Comm Software Engn; IEEE Reliabil Soc; ESTART, IEEE COMPUTER SOC, 10662 LOS VAQUEROS CIRCLE, PO BOX 3014, LOS ALAMITOS, CA 90720-1264 USA, 647–658. https://doi.org/10.1109/ISSRE59848.2023.00026 34th IEEE International Symposium on Software Reliability Engineering (ISSRE), Florence, ITALY, OCT 09-12, 2023.

[11] Moritz Mock, Jorge Melegati, and Barbara Russo. 2024. Generative AI for Test Driven Development: Preliminary Results. *arXiv preprint arXiv:2405.10849* (2024).

[12] Dan North. 2019. Introducing BDD (2006). *Verfügbar unter: http://dannorth.net/introducingbdd* (2019).

[13] Lori Perri. 2023. *Principais tendências estratégicas de segurança cibernética em 2023*. https://www.gartner.com.br/pt-br/artigos/principais-tendencias-estrategicas-de-seguranca-cibernetica-em-2023

[14] Max Schafer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* 50, 1 (JAN 2024), 85–105. https://doi.org/10.1109/TSE.2023.3334955

[15] John Ferguson Smart. 2014. *BDD in Action*. Manning Publications.

[16] Elifrancis Rodrigues Soares. 2006. *Processo de análise de cobertura alinhado ao processo de desenvolvimento de software*. Ph. D. Dissertation. Master's thesis, Informatics Center, Federal University of Pernambuco (UFPE).

[17] Carlos Solis and Wang Xiaofeng. 2011. A study of the characteristics of behaviour driven development. In *Conference on Software Engineering and Advanced Applications (37th EUROMICRO)*. IEEE, Oulu, Finland, 4–13. https://doi.org/10.1109/SEAA.2011.76

[18] Wannita Takerngsaksiri, Rujikorn Charakorn, Chakkrit Tantithamthavorn, and Yuan-Fang Li. 2024. TDD Without Tears: Towards Test Case Generation from Requirements through Deep Reinforcement Learning. *arXiv preprint arXiv:2401.07576* (2024).

[19] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* (2024).

[2] https://github.com/gomesrocha/AutoDevSuite